

本地大模型知识库系列

从入门到精通

AI技术专栏

2026年

目录

1. 本地大模型知识库入门
2. 搭建Ollama环境
3. 构建本地知识库
4. 打造智能问答界面
5. 部署与性能优化
6. 实际应用场景与最佳实践
7. RAG原理
8. 问答接口开发
9. Web界面
10. 性能优化
11. 多模态支持
12. 生产部署

1. 本地大模型知识库入门

本地大模型知识库入门：为什么要自己搭？

本地大模型知识库入门

为什么要自己搭？数据不出域才是真安全

本地大模型知识库系列第01篇 | 讲透RAG原理

你有没有这样的经历：把公司内部文档上传到某个AI聊天工具，第二天就收到安全部门的警告？或者在使用ChatGPT时，犹豫要不要粘贴内部数据，生怕商业机密泄露？

这不是危言听。

"数据上云"在AI时代已经变成了"数据裸奔"

。每一次你把文档喂给云端大模型，都相当于把公司的核心资产交给了一个你无法控制的第三方。

用简单的话解释复杂概念

本地知识库 = 你家的私人图书馆 + 智能图书管理员。所有书都在你家里，管理员（大模型）也在你家里工作，从不把书带出去。

01

云端AI是"数据黑洞"

让我们算一笔账。假设你有1000份内部文档，每份平均1000字。你把它们都上传到某个AI服务来训练专属模型。结果是什么？

1

你的数据变成了别人的训练素材

大多数AI服务商都会保留你的数据用于模型优化，即使他们声称"不会"。你无法验证。

2

你的竞争对手可能获得你的知识

如果你的竞争对手也用同一个服务，他们的模型可能"学到"你的业务知识，这不是科幻小说。

3

你失去了数据主权

服务商可以随时停止服务、提高价格、改变条款，而你束手无策。

数据安全：本地 vs 云端

02

本地部署不是"技术洁癖"

很多人觉得"本地部署"是技术大牛的专利，是不切实际的"技术洁癖"。这是最大的误解

。

今天，搭建一个本地知识库系统的门槛已经降到了历史最低点。你只需要三样东西：

Ollama

—— 一个命令就能下载和运行大模型的工具；

ChromaDB

—— 一个不需要额外配置的轻量级向量数据库；

Python

—— 你可能已经在用的编程语言。

用类比理解抽象概念

RAG（检索增强生成）就像你考试时翻书找答案：先在书里（知识库）找到相关内容，再用自己的话（大模型）组织起来回答老师（用户）。

RAG原理：检索 → 生成 → 回答

03

本地 vs 云端：一张表看清楚

对比维度

本地部署

云端服务

数据安全

数据完全自主控制，零泄露风险

依赖服务商承诺，存在泄露可能

响应速度

毫秒级响应，无网络延

取决于网络和服务器负载

使用成本

一次性硬件投入，后续零费用

按调用量付费，长期成本高

离线可用

完全离线运行

必须联网

定制能力

完全自定义，按需修改

功能受限，依赖服务商更新

04

实战：10行代码体验本地大模型

说了这么多理论，不如来一段实战。在这个系列中，我们会从零开始构建一个完整的本地知识库系统。这里先给你看一下核心代码的骨架：

```
bash
```

```
pip install ollama chromadb sentence-transformers
```

```
python
```

```
import
```

```
ollama
```

```
response
```

```
ollama.chat
```

```
(
```

```
model
```

```
'llama3:8b'
```

```
,
```

```
messages
```

```
{  
  
  role  
  
  :  
  
  'user'  
  
  ,  
  
  '  
  
  content  
  
  :  
  
  '你好'  
  
  }  
  
)  
  
print  
  
(  
  
  response  
  
  [  
  
    'message'  
  
    ]  
  
    'content'  
  
  ])  
  

```

就这么简单。

三行代码，你的第一个本地大模型对话就跑起来了

。后续我们会把这个骨架扩展成一个完整的知识库系统。

教别人是最好的学习方式

这个系列不仅是教程，更是一个学习框架。每一篇文章都配有可运行的代码，你可以跟着做、改一改、教给别人，这样才能真正掌握。

写在最后

本地大模型知识库不是技术炫技，而是企业和个人在AI时代的

生存必需品

。数据安全不是可选的附加功能，而是必须守住的底线。

在接下来的12篇文章中，我们将一起从零搭建这个系统。你不需要是技术专家，但需要有

动手实践的決心

。准备好了吗？

下期預告

第02篇：环境搭建——10分钟跑通本地大模型。我们将安装Ollama，下载第一个大模型（Llama3-8B），并体验命令行对话。

互动：你最关心本地知识库的哪个方面？数据安全、响应速度还是使用成本？欢迎留言讨论。

关注公众号，获取完整代码库和系列文章

系列文章导航

01 | 本地知识库入门——为什么要自己搭？

(你在这里)

02 | 环境搭建——10分钟跑通本地大模型

03 | 工具链整合——Ollama + Python开发环境

04 | 文档处理——把PDF/Word变成AI能看懂的"知识"

05 | 向量数据库——ChromaDB入门

06 | Embedding——把文字变成"数字指纹"

07 | RAG原理——检索增强生成全流程

08 | 问答接口开发——构建你的专属AI助手

09 | Web界面——让知识库"好用"起来

10 | 性能优化——让知识库更快、更准

11 | 多模态支持——图文混合知识库

12 | 生产部署——从开发到上线

2. 搭建Ollama环境

搭建Ollama环境：三步搞定本地大模型

搭建Ollama环境

三步搞定本地大模型

本地大模型知识库系列第02篇 | 环境搭建

上一篇我们讲了为什么要搭建本地知识库，今天直接动手实操。很多人觉得搭建本地大模型门槛很高，需要懂Docker、懂GPU、懂Linux...其实完全不是这样。

Ollama的出现，把本地大模型的门槛降到了最低。

它就像一个"大模型应用商店"，一行命令就能下载和运行任何开源大模型。

01

第一步：安装Ollama

Ollama支持macOS、Linux和Windows，安装方式简单到令人发指：

操作系统

安装方式

macOS

下载

.dmg

双击安装

Linux

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows

下载

.exe

双击安装

安装完成后，打开终端（macOS/Linux）或命令提示符（Windows），输入：


```
bash
```

```
ollama --version
```

如果看到版本号，说明安装成功！

02

第二步：拉取大模型

Ollama支持几十种开源大模型，包括Llama3、Qwen、DeepSeek、Mixtral等。我们先从最经典的Llama3开始：

```
bash
```

```
ollama pull llama3:8b
```

这个命令会下载Meta的Llama3-8B模型，大约4-5GB大小。第一次下载可能需要几分钟，取决于你的网络速度。

用简单的话解释复杂概念

"pull"就像从应用商店下载App，把模型文件从Ollama的服务器下载到你的电脑上。下载完成后，模型就完全属于你了。

如果你想体验更强的中文能力，可以试试阿里的Qwen模型：

```
bash
```

```
ollama pull qwen:7b
```

03

第三步：命令行测试

模型下载完成后，直接在终端里和它对话：

```
bash
```

```
ollama run llama3:8b
```

现在你可以直接在终端里输入问题了，比如：

```
bash
```

```
>> 你好，介绍一下你自己
```

```
我是Llama
```

3，一个由Meta开发的开源大语言模型。我可以帮助你完成各种任务，包括回答问题、生成文本、编写代码等

等。

如果想退出对话，按Ctrl+D即可。

进阶

Python API调用

仅仅在终端里对话还不够酷，我们用Python来调用它：

```
bash
```

```
pip install ollama
```

```
python
```

```
import
```

```
ollama
```

```
# 简单对话
```

```
response
```

```
ollama.chat
```

```
(
```

```
model
```

```
'llama3:8b'
```

```
,
```

```
messages
```

```
{
```

```
'role'
```

```
:
```

```
'user'
```

```
,
```

```
'content'
```

```
:
```

```
'什么是RAG？'
```

```
}
```

```
]
```

```
)
```

```
print
```

```
(
```

```
response
```

```
[
```

```
'message'
```

```
][
```

```
'content'
```

```
])
```

运行这段代码，你就能在Python程序里和本地大模型对话了！

常见问题

1. 第一次运行模型会比较慢，需要加载到内存，后续调用会快很多
2. 如果内存不足，可以尝试更小的模型，如

llama3:70b

需要更多资源

3. 确保你的网络能访问Ollama的模型服务器，必要时配置代理

—— 未完待续 ——

本地大模型知识库系列第02篇

下一篇：构建本地知识库

3. 构建本地知识库

构建本地知识库：从文档到智能问答

构建本地知识库

从文档到智能问答

本地大模型知识库系列第03篇 | RAG实战

前两篇我们讲了为什么要搭建本地知识库，以及如何安装Ollama环境。今天终于到了最核心的环节——构建一个完整的本地知识库系统。

RAG (Retrieval-Augmented Generation)

是构建知识库的核心技术。简单来说，就是把你的文档变成机器能理解的"知识"，然后让大模型基于这些知识来回答问题。

01

知识库架构总览

一个完整的本地知识库系统包含四层架构：

文档层

：存放原始文档，支持PDF、Word、Markdown、网页等多种格式；

向量层

：使用Embedding模型将文本转换为向量，并存入向量数据库；

检索层

：根据用户问题进行语义检索和关键词检索；

应用层

：基于检索结果，让LLM生成最终回答。

02

第一步：文档加载与分割

首先，我们需要把文档加载到系统中。但大模型一次只能处理有限的文本，所以需要把长篇文档分割成小块。

常用的分割方式有三种：按字符数分割、按段落分割、按章节分割。其中按段落分割能较好地保持语义完整性。

```
bash
```

```
pip install langchain chromadb sentence-transformers pypdf
```

```
python
```

```
from
```

```
langchain.document_loaders
```

```
import
```

```
PyPDFLoader
```

```
from
```

```
langchain.text_splitter
```

```
import
```

```
RecursiveCharacterTextSplitter
```

```
# 加载PDF文档
```

```
loader
```

```
PyPDFLoader
```

```
(
```

```
'your_document.pdf'
```

```
)
```

```
documents
```

```
loader
```

```
.
```

```
load
```

```
()
```

```
# 分割文本
```

```
text_splitter
```

```
RecursiveCharacterTextSplitter
```

```
(  
    chunk_size  
  
    500  
  
    ,  
    chunk_overlap  
  
    50  
  
    ,  
    length_function  
  
    len  
  
    ,  
)  
chunks  
  
text_splitter  
.  
split_documents  
  
(  
    documents  
)  
print  
  
(  
    f"分割成 {len(chunks)} 个片段"  
)  

```

用简单的话解释复杂概念

chunk_size=500表示每个片段最多500个字符，chunk_overlap=50表示相邻片段有50个字符的重叠，这样能确保上下文不会被完全切断。

第二步：向量化与存储

文本分割完成后，需要将每个片段转换为向量。这个过程就像是给每段文字"拍照"，把文字变成一串数字，机器可以通过比较这些数字来判断语义相似度。

向量化完成后，需要选择一个向量数据库来存储这些向量。以下是几种常见的选择：

对于本地部署，

Chroma

是最佳选择——轻量级、无需额外部署、API友好。下面是完整的向量化和存储代码：

```
python

from

langchain.vectorstores

import

Chroma

from

langchain.embeddings

import

HuggingFaceEmbeddings

# 初始化Embedding模型

embeddings

HuggingFaceEmbeddings

(

model_name

"all-MiniLM-L6-v2"

)

# 创建向量数据库

vector_store
```

Chroma

.

from_documents

(

documents

chunks

,

embedding

embeddings

,

persist_directory

"./chroma_db"

)

持久化到磁盘

vector_store

.

persist

()

print

(

"向量数据库创建完成！"

)

04

第三步：构建RAG问答系统

现在，文档已经变成了可检索的知识。接下来，我们需要构建一个完整的问答系统，让用户可以基于自己的文

档提问。

```
python
```

```
from
```

```
langchain.chains
```

```
import
```

```
RetrievalQA
```

```
from
```

```
langchain.llms
```

```
import
```

```
Ollama
```

```
# 初始化Ollama LLM
```

```
llm
```

```
Ollama
```

```
(
```

```
model
```

```
"llama3:8b"
```

```
)
```

```
# 加载向量数据库
```

```
vector_store
```

```
Chroma
```

```
.
```

```
persist_directory
```

```
(
```

```
persist_directory
```

```
"/chroma_db"  
  
,  
  
embedding_function
```

```
embeddings
```

```
)
```

```
# 创建检索器
```

```
retriever
```

```
vector_store
```

```
.
```

```
as_retriever
```

```
(
```

```
search_kwargs
```

```
{
```

```
"k"
```

```
:
```

```
3
```

```
}
```

```
)
```

```
# 创建RAG问答链
```

```
qa_chain
```

```
RetrievalQA
```

```
.
```

```
from_chain_type
```

```
(
```

llm

llm

,

chain_type

"stuff"

,

retriever

retriever

,

return_source_documents

True

,

)

提问

query

"请解释文档中的核心概念"

result

qa_chain

({

"query"

:

query

})

```
print  
  
(  
  
"回答:"  
  
)
```

```
print  
  
(  
  
result  
  
[  
  
"result"  
  
])
```

进阶

优化检索效果

想要更好的检索效果，可以尝试以下优化技巧：

优化方法

说明

调整chunk_size

根据文档类型调整，技术文档可以适当增大

使用更好的Embedding模型

尝试bge-base-zh等中文优化模型

调整检索数量k

k值越大，检索结果越全面但可能引入噪声

使用不同的chain_type

map_reduce适合长文档，refine适合需要迭代优化的场景

常见问题

1. Embedding模型下载慢：可以使用国内镜像或提前下载模型文件
2. 向量数据库占用空间大：考使用量化后的模型或清理临时文件

3. 检索结果不准确：尝试调整chunk_size、更换Embedding模型或增加k值
4. 中文支持不好：使用bge-base-zh或text2vec-large-chinese等中文模型

—— 未完待续 ——

本地大模型知识库系列第03篇

下一篇：打造智能问答界面

4. 打造智能问答界面

打造智能问答界面：让你的知识库更易用

打造智能问答界面

让你的知识库更易用

本地大模型知识库系列第04篇 | UI界面

前几篇我们搭建了完整的RAG知识库系统，但目前只能在命令行中使用。今天，我们要给它加上一个漂亮的Web界面，让任何人都能轻松使用。

选择一个合适的UI框架是关键。对于AI应用，有几个特别好用的框架：

对于知识库应用，

Gradio

是最佳选择——简单易用、支持流式输出、一键分享功能。下面我们用Gradio来构建完整的问答界面。

01

界面结构设计

一个完整的聊天界面需要包含以下元素：

02

搭建Gradio聊天界面

bash

pip install gradio

python

import

gradio

as

gr

from

langchain.vectorstores

```
import
Chroma

from
langchain.embeddings

import
HuggingFaceEmbeddings

from
langchain.chains

import
RetrievalQA

from
langchain.llms

import
Ollama

# 初始化组件

embeddings

HuggingFaceEmbeddings

(
model_name

"all-MiniLM-L6-v2"
)

vector_store

Chroma

.

persist_directory
```

```
(  
    persist_directory  
  
    "./chroma_db"  
  
    ,  
  
    embedding_function  
  
    embeddings  
  
)
```

llm

Ollama

```
(  
    model
```

"llama3:8b"

```
)
```

retriever

vector_store

```
.
```

as_retriever

```
(
```

search_kwargs

```
{
```

"k"

```
:
```

3


```
}  
  
)  
  
qa_chain  
  
RetrievalQA  
  
.  
  
from_chain_type  
  
(  
  
llm  
  
llm  
  
,  
  
chain_type  
  
"stuff"  
  
,  
  
retriever  
  
retriever  
  
,  
  
return_source_documents  
  
True  
  
)  
  
# 聊天函数  
  
def  
  
chat  
  
(  
  
message
```

```
,
history
):
result

qa_chain

({
"query"
:
message
})

answer

result

[
"result"
]

return

answer

# 构建界面

with

gr.Blocks

(

title

"本地知识库"

)

as
```

demo

:

gr

.

Markdown

(

"# 本地知识库问答系统"

)

gr

.

ChatInterface

(

chat

,

title

"知识库助手"

,

description

"基于本地大模型的智能问答系统"

)

启动服务

if

__name__

"__main__"

:

```
demo
```

```
.
```

```
launch
```

```
(
```

```
share
```

```
True
```

```
)
```

用简单的话解释复杂概念

`share=True`参数会生成一个临时的公开链接，让你可以分享给朋友测试。如果不需要分享，设置为False即可。

03

实现流式输出

默认情况下，回答是一次性返回的。为了更好的用户体验，我们可以实现流式输出，让回答像打字一样逐字显示。

```
python
```

```
def
```

```
chat_stream
```

```
(
```

```
message
```

```
,
```

```
history
```

```
):
```

```
result
```

```
qa_chain
```

```
({
```

```
"query"
```

```
:  
  
message  
  
})  
  
answer  
  
result  
  
[  
  
"result"  
  
]  
  
# 流式输出  
  
streamed_answer  
  
""  
  
for  
  
char  
  
in  
  
answer  
  
:  
  
streamed_answer  
  
+=  
  
char  
  
yield  
  
streamed_answer  
  
with  
  
gr.Blocks  
  
(  
  
title
```

"本地知识库"

)

as

demo

:

gr

.

Markdown

(

"# 本地知识库问答系统"

)

gr

.

ChatInterface

(

chat_stream

,

title

"知识库助手"

,

description

"基于本地大模型的智能问答系统"

,

streaming

True

)

04

添加文件上传功能

让用户可以直接上传PDF文档来扩展知识库：

```
python
```

```
from
```

```
langchain.document_loaders
```

```
import
```

```
PyPDFLoader
```

```
from
```

```
langchain.text_splitter
```

```
import
```

```
RecursiveCharacterTextSplitter
```

```
# 全局向量数据库实例
```

```
global_vector_store
```

```
None
```

```
def
```

```
load_documents
```

```
(
```

```
files
```

```
):
```

```
global
```

```
global_vector_store
```

```
all_chunks
```

```
[]
```

text_splitter

RecursiveCharacterTextSplitter

(

chunk_size

500

,

chunk_overlap

50

)

for

file

in

files

:

loader

PyPDFLoader

(

file

.

name

)

docs

loader

.

load

()

chunks

text_splitter

.

split_documents

(

docs

)

all_chunks

+=

chunks

global_vector_store

Chroma

.

from_documents

(

documents

all_chunks

,

embedding

embeddings

,

persist_directory

```
"/chroma_db"

)

global_vector_store

.

persist

()

return

f

"成功加载 {len(all_chunks)} 个文档片段！"

def

chat_with_upload

(

message

,

history

):

if

global_vector_store

is

None

:

return

"请先上传文档！"

retriever

global_vector_store

.
```

as_retriever

(

search_kwargs

{

"k"

:

3

}

)

qa_chain

RetrievalQA

.

from_chain_type

(

llm

llm

,

chain_type

"stuff"

,

retriever

retriever

)

result

```
qa_chain
```

```
({
```

```
"query"
```

```
:
```

```
message
```

```
})
```

```
return
```

```
result
```

```
[
```

```
"result"
```

```
]
```

进阶

美化界面

添加自定义主题和样式，让界面更专业：

```
python
```

```
with
```

```
gr.Blocks
```

```
(
```

```
title
```

```
"本地知识库"
```

```
,
```

```
theme
```

```
gr
```

```
.
```

```
themes
```

```
.
Soft
(
primary_hue

"emerald"
,
secondary_hue

"blue"
)
as
demo
:
gr
.
Markdown
"# 本地知识库问答系统"
)
with
gr.Row
(
equal_height

False
):
with
gr.Column
```

```
(
  scale

  1

):

file_upload

gr

.

File

(

  file_types

  [

    ".pdf"

  ],

  file_count

  "multiple"

)

upload_btn

gr

.

Button

(

  "上传并加载文档"

)

upload_status
```

```
gr
.
Textbox
(
label

"状态"
)
with
gr.Column
(
scale

2
):
gr
.
ChatInterface
(
chat_with_upload
,
title

"知识库助手"
,
description

"上传PDF文档，智能问答你的知识"
```

)

upload_btn

.

click

(

load_documents

,

inputs

file_upload

,

outputs

upload_status

)

常见问题

1. 端口被占用：使用`launch(server\_port=7861)`指定其他端口
2. 页面加载慢：首次加载需要初始化Embedding模型，耐心等待
3. share链接无法访问：可能是网络问题，尝试使用本地地址
4. 上传文件过大：使用`max\_file\_size\_mb`参数限制文件大小

—— 未完待续 ——

本地大模型知识库系列第04篇

下一篇：部署与性能优化

5. 部署与性能优化

部署与性能优化：从原型到生产环境

部署与性能优化

从原型到生产环境

本地大模型知识库系列第05篇 | 部署优化

前面四篇我们已经完成了知识库的构建和界面开发，但这些都只是在本地开发环境运行。如果想要让更多人使用，或者需要更高的性能，就需要进行部署和优化。

今天，我们就来聊聊如何将知识库系统从原型部署到生产环境，并进行性能优化。

01

部署方案选择

根据使用场景和需求，我们可以选择不同的部署方案：

02

Docker部署

Docker是最常用的部署方式，可以确保环境一致性。创建Dockerfile：

dockerfile

FROM

python:3.10-slim

WORKDIR

/app

COPY

requirements.txt .

RUN

pip install --no-cache-dir -r requirements.txt

COPY

..

EXPOSE

7860

CMD

```
["python", "app.py"]
```

```
bash
```

```
# 构建镜像
```

```
docker build -t local-kb-app .
```

```
# 运行容器
```

```
docker run -p 7860:7860 local-kb-app
```

03

性能优化策略

性能优化是一个循序渐进的过程，需要先找到瓶颈，再针对性优化：

04

模型量化

模型量化是提升性能最直接有效的方法，可以在牺牲少量精度的情况下大幅降低内存占用和提升推理速度：

```
bash
```

```
# 使用INT4量化模型
```

```
ollama pull llama3:8b-int4
```

```
# 使用Qwen的INT4版本
```

```
ollama pull qwen:7b-int4
```

05

缓存策略

缓存可以显著减少重复计算，提升响应速度：

```
python
```

```
from
```

```
langchain.cache
```

```
import
```

```
InMemoryCache
```

```
from
```

```
langchain.globals
```

```
import
```

```
set_llm_cache
```

```
# 设置内存缓存
```

```
set_llm_cache
```

```
(
```

```
InMemoryCache
```

```
())
```

```
# 或者使用Redis缓存（推荐生产环境）
```

```
# pip install redis
```

```
from
```

```
langchain.cache
```

```
import
```

```
RedisCache
```

```
import
```

```
redis
```

```
redis_client
```

```
redis
```

```
.
```

```
Redis
```

```
(
```

```
host
```

```
"localhost"
```

```
,
```

```
port
```

```
6379
```

```
)
```

```
set_llm_cache
```

```
(
```

```
RedisCache
```

```
(
```

```
redis_=redis_client
```

```
))
```

```
06
```

生产环境架构

对于生产环境，需要更完善的架构设计：

```
python
```

```
import
```

```
uvicorn
```

```
from
```

```
fastapi
```

```
import
```

```
FastAPI
```

```
from
```

```
pydantic
```

```
import
```

```
BaseModel
```

```
app
```

FastAPI

(

title

"本地知识库API"

)

class

QueryRequest

(

BaseModel

):

query

:

str

class

QueryResponse

(

BaseModel

):

answer

:

str

@app.post

(

"/api/chat"

)

def

```
chat
(
  request
  :
  QueryRequest
):
  result

qa_chain
({
  "query"
  :
  request
  .
  query
})
return
  QueryResponse
  (
    answer

    result

    [
      "result"
    ]
  )
  if
  __name__
```

```
"__main__"
```

```
:
```

```
uvicorn.run
```

```
(
```

```
"api:app"
```

```
,
```

```
host
```

```
"0.0.0.0"
```

```
,
```

```
port
```

```
8000
```

```
,
```

```
workers
```

```
4
```

```
)
```

进阶

监控与告警

生产环境需要监控系统运行状态，及时发现和处理问题：

```
python
```

```
import
```

```
time
```

```
import
```

```
psutil
```

```
def
```

```
get_system_metrics
```

():

cpu_percent

psutil

.

cpu_percent

()

memory_percent

psutil

.

virtual_memory

()

.

percent

return

{

"cpu_percent"

:

cpu_percent

,

"memory_percent"

:

memory_percent

,

"timestamp"

:

time

.

time

()

}

@app.get

(

"/api/health"

)

def

health_check

():

metrics

get_system_metrics

()

return

{

"status"

:

"healthy"

,

"metrics"

:

metrics

}

常见问题

1. Docker内存不足：增加容器内存限制，使用

参数

2. 模型加载慢：使用较小的模型或预加载到内存

3. 并发请求超时：增加超时时间或使用异步处理

4. 缓存失效：设置合理的缓存过期时间

—— 未完待续 ——

本地大模型知识库系列第05篇

下一篇：实际应用场景与最佳实践

6. 实际应用场景与最佳实践

实际应用场景与最佳实践：系列完结篇

实际应用场景与最佳实践

系列完结篇

本地大模型知识库系列第06篇 | 总结与展望

经过前五篇的学习，我们已经从零基础搭建了一个完整的本地大模型知识库系统。今天是系列的最后一篇，我们来聊聊这个系统的实际应用场景、最佳实践，以及未来的发展趋势。

01

应用场景

本地大模型知识库的应用场景非常广泛，涵盖个人、企业、教育等多个领域：

02

最佳实践

在实际使用中，遵循以下最佳实践可以让你的知识库系统更加稳定和高效：

03

企业级部署方案

如果需要在企业环境中部署，需要考更多的安全性和可扩展性：

04

知识管理流程

一个健康的知识库需要持续的维护和更新：

05

未来发展趋势

本地大模型知识库领域正在快速发展，未来有几个值得关注的方向：

06

系列总结

回顾整个系列，我们从基础概念开始，一步步构建了一个完整的本地大模型知识库系统：

系列回顾

第01篇

：为什么需要本地知识库——数据安全、隐私保护、离线使用

第02篇

：搭建Ollama环境——三步搞定本地大模型

第03篇

：构建本地知识库——从文档到智能问答

第04篇

：打造智能问答界面——让你的知识库更易用

第05篇

：部署与性能优化——从原型到生产环境

第06篇

：应用场景与最佳实践——系列完结篇

进阶

快速启动模板

为了方便大家快速上手，这里提供一个完整的启动脚本：

```
bash
```

```
# 一键部署脚本
```

```
# 1. 安装Ollama
```

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
# 2. 拉取模型
```

```
ollama pull llama3:8b
```

```
# 3. 创建项目目录
```

```
mkdir local-kb && cd local-kb
```

```
# 4. 安装依赖
```

```
pip install langchain chromadb sentence-transformers gradio pypdf
```

5. 创建应用

```
cat > app.py << 'EOF'
```

```
import gradio as gr
```

```
from langchain.vectorstores import Chroma
```

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
from langchain.chains import RetrievalQA
```

```
from langchain.llms import Ollama
```

```
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

```
vector_store = Chroma(persist_directory="./chroma_db", embedding_function=embeddings)
```

```
llm = Ollama(model="llama3:8b")
```

```
retriever = vector_store.as_retriever(search_kwargs={"k": 3})
```

```
qa_chain = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever)
```

```
def chat(message, history):
```

```
    result = qa_chain({"query": message})
```

```
    return result["result"]
```

```
with gr.Blocks(title="本地知识库") as demo:
```

```
    gr.Markdown("# 本地知识库问答系统")
```

```
    gr.ChatInterface(chat, title="知识库助手")
```

```
if __name__ == "__main__":
```

```
    demo.launch(share=False)
```

```
EOF
```

6. 启动服务

```
python app.py
```

—— 系列完结 ——

本地大模型知识库系列第06篇（完结篇）

感谢一路陪伴！如有问题欢迎交流

7. RAG原理

RAG原理——检索增强生成全流程

RAG原理

检索增强生成全流程

本地大模型知识库系列第07篇 | RAG核心

前面我们已经学会了文档处理、向量数据库和Embedding，现在终于到了最核心的环节——RAG（Retrieval-Augmented Generation，检索增强生成）。

RAG的核心思想非常简单

：在让大模型回答问题之前，先从知识库中检索相关的文档片段，然后把这些文档和问题一起交给大模型，让它基于这些文档来回答。

01

RAG vs 纯LLM

为什么需要RAG？直接让大模型回答不行吗？来看一下对比：

02

RAG核心流程详解

RAG的完整流程可以分为三个阶段：

第一阶段：检索

1. 用户提问 → 2. 问题向量化 → 3. 向量检索 → 4. 获取相关文档片段

第二阶段：构建

5. 将检索到的文档和原始问题一起构建成Prompt

第三阶段：生成

6. 将Prompt交给大模型 → 7. LLM生成回答 → 8. 返回结果

03

Prompt工程

RAG的效果很大程度上取决于Prompt的质量。一个好的Prompt应该包含以下几个要素：

```
python
```

```
def
```

```
build_prompt
```

```
(
```

```
query
```

```
,
```

```
context
```

```
):
```

```
prompt
```

```
"""你是一个专业的知识库助手，擅长基于给定的文档内容回答问题。
```

```
## 角色设定
```

```
## 参考文档
```

```
{context}
```

```
## 用户问题
```

```
{query}
```

```
## 回答要求
```

```
return
```

```
prompt
```

```
.
```

```
format
```

```
(
```

```
context
```

context

,

query

query

)

04

完整RAG实现

python

from

langchain.chains

import

RetrievalQA

from

langchain.llms

import

Ollama

from

langchain.vectorstores

import

Chroma

from

langchain.embeddings

import

HuggingFaceEmbeddings

初始化组件

embeddings


```
HuggingFaceEmbeddings
```

```
(
```

```
model_name
```

```
"all-MiniLM-L6-v2"
```

```
)
```

```
vector_store
```

```
Chroma
```

```
.
```

```
persist_directory
```

```
(
```

```
persist_directory
```

```
"./chroma_db"
```

```
,
```

```
embedding_function
```

```
embeddings
```

```
)
```

```
llm
```

```
Ollama
```

```
(
```

```
model
```

```
"llama3:8b"
```

```
)
```

```
# 创建检索器
```

retriever

vector_store

.

as_retriever

(

search_kwargs

{

"k"

:

3

}

)

创建RAG链

qa_chain

RetrievalQA

.

from_chain_type

(

llm

llm

,

chain_type

"stuff"

,

```
retriever
```

```
retriever
```

```
,
```

```
return_source_documents
```

```
True
```

```
)
```

```
# 测试问答
```

```
query
```

```
"请解释什么是RAG ? "
```

```
result
```

```
qa_chain
```

```
({
```

```
"query"
```

```
:
```

```
query
```

```
})
```

```
print
```

```
(
```

```
"回答:"
```

```
)
```

```
print
```

```
(
```

```
result
```

```
[
```

```
"result"

])

print

(

"\n引用来源:"

)

for

doc

in

result

[

"source_documents"

]:

print

(

doc

.

metadata

.

get

(

"source"

))
```

常见问题

1. 检索结果不相关：调整chunk_size或更换Embedding模型
2. 回答质量差：优化Prompt或使用更大的模型

3. 回答速度慢：使用更小的模型或添加缓存

4. 上下文超限：减少返回的文档数量(k值)

—— 未完待续 ——

本地大模型知识库系列第07篇

下一篇：问答接口开发——构建你的专属AI助手

8. 问答接口开发

问答接口开发——构建你的专属AI助手

问答接口开发

构建你的专属AI助手

本地大模型知识库系列第08篇 | 接口开发

上一篇我们学习了RAG的核心原理，现在我们要把这个能力封装成一个可调用的接口，方便其他应用使用。

01

封装问答函数

```
python
```

```
import
```

```
json
```

```
import
```

```
time
```

```
from
```

```
langchain.chains
```

```
import
```

```
RetrievalQA
```

```
from
```

```
langchain.llms
```

```
import
```

```
Ollama
```

```
from
```

```
langchain.vectorstores
```

```
import
```

```
Chroma
```

```
from
langchain.embeddings

import
HuggingFaceEmbeddings

class
KnowledgeBaseQA

:

def

__init__

(

self

,

model_name

"llama3:8b"

,

db_path

"./chroma_db"

):

self

.

embeddings

HuggingFaceEmbeddings

(

model_name
```

"all-MiniLM-L6-v2"

)

self

.

vector_store

Chroma

.

persist_directory

(

persist_directory

db_path

,

embedding_function

self

.

embeddings

)

self

.

llm

Ollama

(

model

model_name


```
)

self
.

retriever

self
.

vector_store

.

as_retriever

(

search_kwargs

{

"k"

:

3

}

)

self
.

qa_chain

RetrievalQA

.

from_chain_type

(

llm
```

```
self
.
llm
,
chain_type

"stuff"
,
retriever

self
.
retriever
,
return_source_documents

True
)

def
answer
(
self
,
query
,
top_k

3
```

```
):  
  
start_time  
  
time  
  
.  
  
time  
  
()  
  
try  
  
:  
  
result  
  
self  
  
.  
  
qa_chain  
  
({  
  
"query"  
  
:  
  
query  
  
})  
  
sources  
  
[]  
  
for  
  
doc  
  
in  
  
result  
  
[  
  
"source_documents"
```

]:

sources

.

append

{

"source"

:

doc

.

metadata

.

get

(

"source"

),

"page"

:

doc

.

metadata

.

get

(

"page"

,

0

```
),  
  
"score"  
  
:  
  
doc  
  
.  
  
metadata  
  
.  
  
get  
  
(  
  
"score"  
  
,  
  
"N/A"  
  
)  
  
})  
  
response_time  
  
  
  
time  
  
.  
  
time  
  
()  
  
  
start_time  
  
return  
  
{  
  
"success"  
  
:  
  
True
```

```
,  
"answer"  
:  
result  
[  
"result"  
],  
"sources"  
:  
sources  
,  
"response_time"  
:  
round  
(  
response_time  
,  
2  
),  
"model"  
:  
self  
.  
llm  
.  
model
```

```
}  
  
except  
Exception  
  
as  
  
e  
  
:  
  
return  
  
{  
  
"success"  
  
:  
  
False  
  
,  
  
"error"  
  
:  
  
str  
  
(  
  
e  
  
),  
  
"answer"  
  
:  
  
""  
  
,  
  
"sources"  
  
:  
  
[]  
  
}
```

添加引用标注

让用户知道答案来自哪份文档，增加回答的可信度：

```
python
```

```
def
```

```
add_citations
```

```
(
```

```
answer
```

```
,
```

```
sources
```

```
):
```

```
citation_text
```

```
"\n\n## 引用来源\n"
```

```
for
```

```
i
```

```
,
```

```
source
```

```
in
```

```
enumerate
```

```
(
```

```
sources
```

```
,
```

```
1
```

```
):
```

```
source_name
```


source

.

get

(

"source"

,

"未知来源"

)

page

source

.

get

(

"page"

,

""

)

citation_text

+=

f

"{i}. [{source_name}]"

if

page

:

citation_text

+=

f

"(第{page}页)"

citation_text

+=

"\n"

return

answer

+

citation_text

03

测试问答接口

python

初始化问答助手

qa

KnowledgeBaseQA

()

测试问题

query

"请解释什么是RAG ? "

result

qa

.

answer

(

query

```
)  
  
# 输出结果  
  
if  
  
result  
  
[  
  
"success"  
  
]:  
  
print  
  
(  
  
"回答:"  
  
)  
  
print  
  
(  
  
result  
  
[  
  
"answer"  
  
])  
  
print  
  
(  
  
"\n引用来源:"  
  
)  
  
for  
  
src  
  
in  
  
result  
  
[
```

```
"sources"

]:

print

(

f"– {src['source']}"

)

print

(

f

"\n响应时间: {result['response_time']}秒"

)

else

:

print

(

f

"错误: {result['error']}"

)
```

常见问题

1. 文档元数据缺失：确保加载文档时保留metadata信息
2. 引用标注格式混乱：统一引用格式模板
3. 响应时间过长：添加进度回调或异步处理
4. 错误信息不友好：统一错误格式和提示

—— 未完待续 ——

本地大模型知识库系列第08篇

下一篇：Web界面——让知识库"好用"起来

9. Web界面

Web界面——让知识库"好用"起来

Web界面

让知识库"好用"起来

本地大模型知识库系列第09篇 | 界面开发

上一篇我们封装了问答接口，现在需要一个友好的Web界面让用户能够轻松使用。我们将使用Gradio来快速构建一个美观的聊天界面。

01

安装Gradio

bash

安装Gradio

pip install gradio

02

创建聊天界面

python

import

gradio

as

gr

from

knowledge_base_qa

import

KnowledgeBaseQA

初始化问答助手

qa

KnowledgeBaseQA

()

def

chat

(

message

,

history

):

result

qa

.

answer

(

message

)

if

result

[

"success"

]:

answer

result

[

"answer"

]

if

result

[

"sources"

]:

answer

+=

"\n\n 参考来源:"

for

src

in

result

[

"sources"

]:

answer

+=

f

"\n- {src['source']}"

answer

+=

f

"\n\n 响应时间: {result['response_time']}秒"

return

answer

```
else

:

return

f" 出错了: {result['error']}"

# 创建Gradio界面

with

gr

.

Blocks

()

as

demo

:

gr

.

Markdown

(

"""

# 本地知识库问答

基于本地大模型的智能问答系统

"""

)

chatbot

gr

.

Chatbot
```



```
(
  height

  500

  ,
  avatar_images

  (
    None

    ,
    "https://img.icons8.com/color/48/000000/bot.png"
  )
)

msg

gr

.
Textbox

(
  label

  "输入问题"

  ,
  placeholder

  "请输入您的问题..."

  ,
  lines
```

2

)

submit_btn

gr

.

Button

(

"发送"

,

variant

"primary"

)

clear_btn

gr

.

Button

(

"清空对话"

)

submit_btn

.

click

(

chat

,

inputs

[

msg

,

chatbot

],

outputs

chatbot

)

msg

.

submit

(

chat

,

inputs

[

msg

,

chatbot

],

outputs

chatbot

)

clear_btn

.

click

(

lambda

:

None

,

None

,

chatbot

,

queue

False

)

启动服务

if

__name__

"__main__"

:

demo

.

launch

(

server_name

"0.0.0.0"

,

server_port

7860

,

share

False

)

03

添加文档上传功能

让用户可以随时上传新文档到知识库：

python

from

langchain.document_loaders

import

PyPDFLoader

from

langchain.text_splitter

import

RecursiveCharacterTextSplitter

def

upload_documents

(

files

):

documents

[]

for

file

in

files

:

loader

PyPDFLoader

(

file

.

name

)

docs

loader

.

load

()

documents

.

extend

(

docs

)

text_splitter

RecursiveCharacterTextSplitter

(

chunk_size

500

,

chunk_overlap

50

)

splits

text_splitter

.

split_documents

(

documents

)

qa

.

vector_store

.

add_documents

(

splits

)

```
return

f

"""成功上传并处理{len(files)}个文档，共{len(splits)}个片段。"""

# 在界面中添加文件上传组件

file_upload

gr

.

File

(

label

"上传文档"

,

file_types

[

".pdf"

,

".txt"

,

".md"

],

file_count

"multiple"

)

upload_btn
```



```
gr
.
Button
(
"上传到知识库"
)
upload_status
```

```
gr
.
Textbox
(
label
"上传状态"
,
visible
False
)
upload_btn
```

```
.
click
(
upload_documents
,
inputs
```

```
file_upload
```

```
,
```

```
outputs
```

```
upload_status
```

```
)
```

常见问题

1. 界面无法访问：检查防火墙和端口配置
2. 上传文件失败：检查文件大小和格式限制
3. 内存占用过高：添加文件大小限制和清理机制
4. 多人并发问题：启用Gradio的queue机制

—— 未完待续 ——

本地大模型知识库系列第09篇

下一篇：性能优化——让知识库更快、更准

10. 性能优化

性能优化——让知识库更快、更准

性能优化

让知识库更快、更准

本地大模型知识库系列第10篇 | 性能优化

上一篇我们创建了Web界面，现在需要对系统进行性能优化，让知识库运行更快、回答更准确。性能优化涉及多个方面，包括模型量化、缓存策略、检索优化等。

01

模型量化

使用量化后的模型可以显著减少内存占用和推理时间：

```
bash
```

```
# 使用4-bit量化模型
```

```
ollama run llama3:8b-instruct-q4_K_M
```

```
# 使用8-bit量化模型（平衡性能和质量）
```

```
ollama run llama3:8b-instruct-q8_0
```

02

添加缓存机制

对相似问题进行缓存，避免重复计算：

```
python
```

```
import
```

```
hashlib
```

```
import
```

```
json
```

```
from
```

```
functools
```

```
import
lru_cache

class
CachedQA
:
def
__init__(
self,
cache_size

1000
):
self
.qa

KnowledgeBaseQA
()
self
.cache

{}

self
.cache_size
```

cache_size

def

_get_cache_key

(

self

,

query

):

return

hashlib

.

md5

(

query

.

encode

())

.

hexdigest

()

def

answer

(

self

,

query

```
):  
  
key  
  
self  
  
.  
  
_get_cache_key  
  
(  
  
query  
  
)  
  
if  
  
key  
  
in  
  
self  
  
.  
  
cache  
  
:  
  
result  
  
self  
  
.  
  
cache  
  
[  
  
key  
  
]  
  
result  
  
[  
  
"response_time"
```

```
] =  
  
0.01  
  
return  
  
result  
  
result  
  
  
self  
  
.  
  
qa  
  
.  
  
answer  
  
(  
  
query  
  
)  
  
if  
  
len  
  
(  
  
self  
  
.  
  
cache  
  
) >=  
  
self  
  
.  
  
cache_size  
  
:  
  
oldest_key
```

```
min
(
self
.
cache
.keys
())
del
self
.
cache
[
oldest_key
]
self
.
cache
[
key
] =
result
return
result
03
```

检索优化

通过调整检索参数提高准确性和速度：


```
python

def
optimize_retrieval
(
vector_store
,
k

5
,
score_threshold

0.7
):
retriever

vector_store
.
as_retriever
(
search_kwargs

{
"k"

:
k
,
"score_threshold"
```

```
:  
  
score_threshold  
  
}  
  
)  
  
return  
  
retriever  
  
# 使用更精确的检索策略  
  
retriever  
  
  
optimize_retrieval  
  
(  
  
qa  
  
.  
  
vector_store  
  
,  
  
k  
  
4  
  
,  
  
score_threshold  
  
0.75  
  
)  
  
# 使用MMR (Maximal Marginal Relevance) 提高多样性  
  
retriever  
  
  
vector_store  
  
.
```

```
as_retriever  
  
(  
  
    search_type  
  
    "mmr"  
  
    ,  
  
    search_kwargs  
  
    {  
  
        "k"  
  
        :  
  
        5  
  
        ,  
  
        "fetch_k"  
  
        :  
  
        20  
  
    }  
  
)
```

04

异步处理

使用异步处理提高并发能力：

```
python  
  
import  
  
asyncio  
  
class  
  
AsyncQA  
  
:
```

```
def
__init__
(
self
):
self
.
qa

KnowledgeBaseQA
()

async
def
answer_async
(
self
,
query
):
loop

asyncio
.
get_event_loop
()
return
await
```

```
loop

·

run_in_executor

(

None

,

self

·

qa

·

answer

,

query

)

# 并发处理多个查询

async

def

process_batch

(

queries

):

    async_qa

    AsyncQA

()

tasks
```

```
[
    async_qa
    .
    answer_async
    (
        q
    )
    for
        q
    in
        queries
]

return

await

asyncio
.

gather
(
    *
    tasks
)
```

常见问题

1. 量化模型效果差：尝试更高精度的量化或调整prompt
2. 缓存命中率低：优化缓存键的生成策略
3. 检索结果不准确：调整k值和score_threshold
4. 并发性能差：使用异步处理和线程池

—— 未完待续 ——

本地大模型知识库系列第10篇

下一篇：多模态支持——图文混合知识库

11. 多模态支持

多模态支持——图文混合知识库

多模态支持

图文混合知识库

本地大模型知识库系列第11篇 | 多模态

上一篇我们学习了性能优化，现在让知识库支持图文混合内容。很多文档不仅包含文字，还包含图表、图片等，我们需要让系统能够理解和处理这些视觉信息。

01

安装多模态依赖

```
bash
```

```
# 安装多模态处理库
```

```
pip install pillow pytesseract transformers
```

```
# 安装OCR工具（用于图片文字识别）
```

```
# macOS
```

```
brew install tesseract
```

```
# Ubuntu
```

```
sudo apt-get install tesseract-ocr
```

02

图片文字识别

使用OCR技术提取图片中的文字信息：

```
python
```

```
import
```

```
pytesseract
```

```
from
```

```
PIL
```



```
import
Image
def
extract_text_from_image
(
image_path
):
image

Image
.
open
(
image_path
)
custom_config

r'--oem 3 --psm 6'
text

pytesseract
.
image_to_string
(
image
,
lang
```

```
'chi_sim+eng'
```

```
,
```

```
config
```

```
custom_config
```

```
)
```

```
return
```

```
text
```

```
# 使用示例
```

```
image_text
```

```
extract_text_from_image
```

```
(
```

```
"document.png"
```

```
)
```

```
print
```

```
(
```

```
"图片中的文字:"
```

```
)
```

```
print
```

```
(
```

```
image_text
```

```
)
```

```
03
```

```
图像理解与描述
```

```
使用视觉语言模型（VLM）理解和描述图片内容：
```

```
python
```

```
import
ollama

def
describe_image
(
image_path
,
prompt

"请描述这张图片的内容"

):
with
open
(
image_path
,
'rb'
)
as
f
:
image_data

f
.
read
()
```

response

ollama

.

chat

(

model

"llava"

,

messages

"role"

:

"user"

,

"content"

:

prompt

,

"images"

:

[

image_data

]

}]

)

return

```
response
```

```
[
```

```
"message"
```

```
][
```

```
"content"
```

```
]
```

```
# 使用示例
```

```
description
```

```
describe_image
```

```
(
```

```
"chart.png"
```

```
)
```

```
print
```

```
(
```

```
"图片描述:"
```

```
)
```

```
print
```

```
(
```

```
description
```

```
)
```

```
04
```

```
图文混合索引
```

```
将图片描述和文字一起存入向量数据库：
```

```
python
```

```
from
```

```
langchain.schema
```

```
import
Document

def
process_multimodal_document
(
text_content
,
image_paths
):
documents

[]
documents
.
append
(
Document
(
page_content

text_content

,
metadata

"type"

:

"text"

}
)
```

```
))

for
img_path
in
image_paths
:
ocr_text

extract_text_from_image
(
img_path
)
img_description

describe_image
(
img_path
)
combined_text

f"图片内容描述: {img_description}\n图片中的文字: {ocr_text}"

documents
.
append
(
Document
(
page_content
```

combined_text

,

metadata

"type"

:

"image"

,

"source"

:

img_path

}

))

return

documents

使用示例

text

"这是一份产品说明书，包含多个图表和图片。"

images

[

"chart1.png"

,

"screenshot.png"

]

docs


```
process_multimodal_document
```

```
(
```

```
text
```

```
,
```

```
images
```

```
)
```

常见问题

1. OCR识别不准确：尝试调整图片分辨率和对比度
2. 图像模型加载失败：检查模型是否已下载
3. 处理速度慢：图像理解比纯文本慢，需要耐心等待
4. 内存占用高：处理大图片时注意内存管理

—— 未完待续 ——

本地大模型知识库系列第11篇

下一篇：生产部署——从开发到上线

12. 生产部署

生产部署——从开发到上线

生产部署

从开发到上线

本地大模型知识库系列第12篇 | 生产部署

这是系列的最后一篇，我们将学习如何将本地大模型知识库部署到生产环境。生产部署需要考定性、安全性、可扩展性等多个方面。

01

使用Docker容器化

创建Dockerfile来构建镜像：

dockerfile

使用Python 3.11基础镜像

FROM

python:3.11-slim

设置工作目录

WORKDIR

/app

安装系统依赖

RUN

apt-get update && apt-get install -y \

build-essential \

gcc \

git \

&& rm -rf /var/lib/apt/lists/*

复制依赖文件

COPY

requirements.txt .

安装Python依赖

RUN

pip install --no-cache-dir -r requirements.txt

复制应用代码

COPY

..

设置环境变量

ENV

MODEL_NAME=llama3:8b

ENV

DB_PATH=/app/chroma_db

暴露端口

EXPOSE

7860

启动命令

CMD

["python", "app.py"]

yaml

docker-compose.yml

version

:

'3.8'

services

:

knowledge-base

:

build

:

.

ports

:

"7860:7860"

volumes

:

./chroma_db:/app/chroma_db

./documents:/app/documents

environment

:

MODEL_NAME=llama3:8b

LOG_LEVEL=INFO

restart

:

unless-stopped

deploy

:

resources

:

reservations

```
:
devices

:

driver

:

nvidia

count

:

1

capabilities
```

```
:
[
gpu
]
02
```

配置Nginx反向代理

使用Nginx提供静态文件服务和反向代理：

```
nginx

# /etc/nginx/sites-available/knowledge-base

server

{

listen

80;

server_name

kb.example.com;

location
```

```
/ {  
  
    proxy_pass  
    http://localhost:7860;  
  
    proxy_set_header  
  
    Host  
  
    $host;  
  
    proxy_set_header  
  
    X-Real-IP  
  
    $remote_addr;  
  
    proxy_set_header  
  
    X-Forwarded-For  
  
    $proxy_add_x_forwarded_for;  
  
    proxy_set_header  
  
    X-Forwarded-Proto  
  
    $scheme;  
  
    # WebSocket支持  
  
    proxy_http_version  
  
    1.1;  
  
    proxy_set_header  
  
    Upgrade  
  
    $http_upgrade;  
  
    proxy_set_header  
  
    Connection  
  
    "upgrade";  
}
```

```
# 配置SSL（使用Let's Encrypt）
```

```
listen

443 ssl;

ssl_certificate

/etc/letsencrypt/live/kb.example.com/fullchain.pem;

ssl_certificate_key

/etc/letsencrypt/live/kb.example.com/privkey.pem;

}
```

03

添加日志和监控

添加日志记录和健康检查：

```
python
```

```
import
```

```
logging
```

```
from
```

```
flask
```

```
import
```

```
Flask
```

```
# 配置日志
```

```
logging
```

```
.
```

```
basicConfig
```

```
(
```

```
level
```

```
logging
```

```
.
```

```
INFO
```

```
,  
  
format  
  
'%(asctime)s - %(name)s - %(levelname)s - %(message)s'  
  
,  
  
handlers  
  
logging  
  
.  
  
FileHandler  
  
(  
  
'app.log'  
  
),  
  
logging  
  
.  
  
StreamHandler  
  
()  
  
]  
  
)  
  
logger  
  
logging  
  
.  
  
getLogger  
  
(  
  
__name__  
  
)  
  
# 添加健康检查端点
```


app

Flask

(

__name__

)

@app

.

route

(

'/health'

)

def

health_check

():

try

:

qa

KnowledgeBaseQA

()

return

{

"status"

:

"healthy"

}

```
except
```

```
Exception
```

```
as
```

```
e
```

```
:
```

```
logger
```

```
.
```

```
error
```

```
(
```

```
f"Health check failed: {e}"
```

```
)
```

```
return
```

```
{
```

```
"status"
```

```
:
```

```
"unhealthy"
```

```
},
```

```
500
```

```
04
```

系列总结

恭喜你完成了整个系列！让我们回顾一下所学内容：

系列回顾

-

第01篇

：了解了本地大模型知识库的价值和优势

-

第02篇

：搭建了Ollama环境并运行了第一个模型

-

第03篇

：构建了基于RAG的本地知识库

-

第04篇

：使用Gradio创建了Web界面

-

第05篇

：学习了系统部署与性能优化

-

第06篇

：了解了应用场景与最佳实践

-

第07篇

：深入理解了RAG的核心原理

-

第08篇

：封装了问答接口

-

第09篇

：完善了Web界面功能

-

第10篇

：实现了多种性能优化策略

-

第11篇

：添加了多模态支持

-

第12篇

：完成了生产环境部署

通过这个系列，你已经掌握了从零开始构建一个完整的本地大模型知识库系统的全部技能。希望这些知识能帮助你在实际工作中更好地应用AI技术！

常见问题

1. Docker镜像太大：使用多阶段构建减少镜像大小
2. GPU资源不足：考使用云GPU或模型量化
3. SSL证书问题：使用Certbot自动管理证书
4. 服务宕机：配置自动重启和监控告警

—— 系列完结 ——

本地大模型知识库系列第12篇（完）

感谢您的阅读！